

AI FitTrack API

Intelligent fitness tracking system with AI-powered workout recommendations

Personalized fitness guidance for smarter training, better results, and lasting progress.

TEAM AI FITTRACK



Daniya Fathima.B
Team Leader



Kathija Rayhana.R
Team Member



Aysha Mafazha.A.S
Team Member



Faiza Fathima.I
Team Member

Tech Stack



Runtime

Node.js

Runs the backend server and executes the application logic.



Framework

Express.js

Handles API routes, middleware, and server-side request processing.



Database

MongoDB

Stores user, workout, and recommendation data in a flexible NoSQL format.



ODM

Mongoose

Defines schemas and manages structured access to MongoDB data.



AI

Google Gemini API

Powers AI-based fitness insights and personalized recommendations.



Authentication

JWT

Secures user sessions with token-based authentication.

bcryptjs

Hashes passwords before saving them to protect user credentials.



Logging

Morgan

Logs incoming API requests to help monitor and debug the backend.



API Testing

Postman

Used to test API endpoints and verify responses during development.



Environment Management

dotenv

Loads environment variables such as secrets, API keys, and database URLs from a .env file.

MVC Architecture

The project follows a clean **MVC-style backend structure** for maintainability and scalability.

📁 ARCHITECTURE

```
FitTrack/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── workoutController.js
│   │   └── aiController.js
│   ├── middleware/
│   │   ├── auth.js
│   │   └── errorHandler.js
│   ├── models/
│   │   ├── User.js
│   │   └── Workout.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── workoutRoutes.js
│   │   ├── aiRoutes.js
│   │   └── index.js
│   ├── services/
│   │   └── geminiService.js
│   └── app.js
├── .env
├── .env.example
├── package.json
└── FitTrack.postman_collection.json
```

Models

Manage **User** and **Workout** data schemas

Controllers

Handle all application and API logic

Routes

Define all API endpoints and mappings

Middleware

JWT authentication and error handling

Services

Connects the app with **Google Gemini AI**

Config

Handles MongoDB database connection via `db.js`

Getting Started

Prerequisites

Node.js v20+

MongoDB Local or Atlas

Google Gemini API Key

Obtain from **Google AI Studio**

Installation

1. Clone the repository to your local machine.
2. Open the project folder in **VS Code** or your terminal.
3. Run `npm install` to install all required dependencies.
4. Create a `.env` file in the project root.
5. Add the required environment variables: `MONGODB_URI`, `GEMINI_API_KEY`, `JWT_SECRET`, and `PORT`.

Example:

```
MONGODB_URI=your_mongodb_connection_string
GEMINI_API_KEY=your_gemini_api_key
JWT_SECRET=your_secret_key
PORT=5000
```

Once your environment is configured, you're ready to start building and running the app.

Running the Server & API Specifications

▶ Start the Application

```
npm run dev
```

or

```
npm start
```

API runs on:

```
http://localhost:5000
```



API Response Format

All endpoints return responses in **JSON** format.



Authentication

Protected routes require a **Bearer token** in the **Authorization** header.

```
Authorization: Bearer <your_token>
```



Error Handling

Use standard **HTTP status codes** for errors, along with a clear error message in the JSON response.

Examples include **400** for bad requests, **401** for unauthorized access, **404** for missing resources, and **500** for server errors.

Authentication & Workout Management APIs

Authentication Endpoints

POST /api/auth/register

Register a new user with name, email, and password.

POST /api/auth/login

Log in with email and password to receive a JWT token.

GET /api/auth/profile

Protected route. Returns the authenticated user's profile details.

Workout Management — /api/workouts

All workout routes are **JWT-authenticated** and **user-scoped**.

Method	Endpoint	Description
GET	/api/workouts	Get all workouts
POST	/api/workouts	Create a workout with name, category, duration, calories, and date
PUT	/api/workouts/:id	Update a workout by ID
DELETE	/api/workouts/:id	Delete a workout by ID

Workout Search & AI Features

Workout Search

GET /api/workouts/search?q=

Case-insensitive regex search across:

- Workout name
- Workout category
- Workout date (standard date pattern)

GET /api/workouts/search?q=running

GET /api/workouts/search?q=2026-06-26

EXAMPLE REQUESTS

AI Features — Powered by Google Gemini AI (JWT required)

1

POST /api/ai/recommend

Protected. Sends the user's recent workouts to Gemini AI and returns personalized workout recommendations based on their activity, goals, and progress.

This helps users get tailored guidance instead of generic advice.

2

POST /api/ai/analyze

Protected. Analyzes workout patterns and returns actionable insights about consistency, volume, and possible areas for improvement.

This gives users a smarter view of their training habits and progress over time.

The AI features make the app more helpful by turning workout history into personalized recommendations and easy-to-understand insights.

Postman Collection & Complete System Workflow

Postman Collection Setup

File: FitTrack.postman_collection.json

Import it into Postman via **Import** → **Upload Files** → select the collection JSON.

Endpoints are grouped by category for easy testing:

- **Authentication** — Register, Login, Get Profile
- **Workouts** — Add, View, Update, Delete
- **Search** — Search workouts by name, category, or date
- **AI Features** — Workout Recommendation and Fitness Insights

1

Automatic Token Handling

Register/Login scripts auto-store JWT

2

Automatic Workout ID

Add Workout auto-stores ID as For Retrieve / Update / Delete

3

Bearer Authentication

Inherited dynamically from collection variables

Environment Variables

Create a Postman environment and define these variables before running requests:

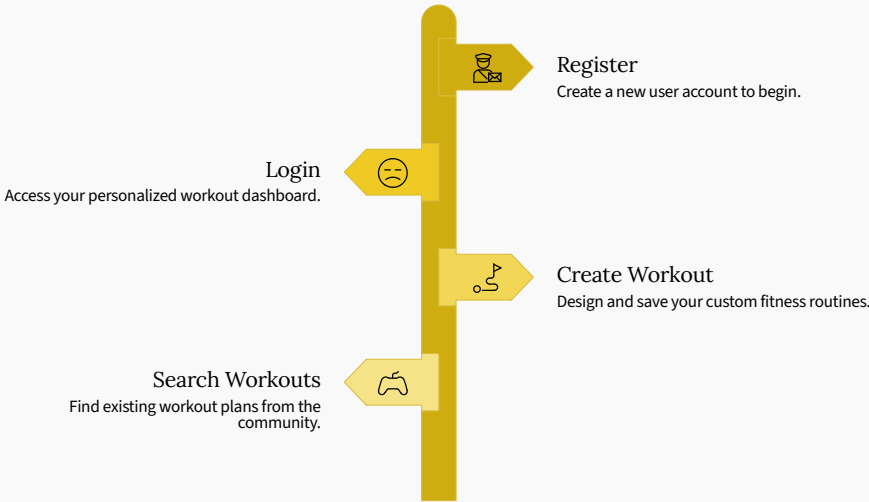
- `base_url` — your API root URL, for example `http://localhost:5000`
- `token` or `jwt_token` — set automatically after login, then reused in the Authorization header

Recommended request order for testing:

1. **Register** a new user
2. **Login** and confirm the JWT is saved
3. **Create Workout** to save a workout ID
4. **Search Workouts** using name, category, or date filters
5. **Get AI Recommendations** based on user inputs
6. **Analyze Patterns** using workout history and summary metrics

This collection is designed as a practical testing guide so you can validate authentication, CRUD operations, search, and AI endpoints in a single workflow.

Complete System Workflow



For AI features: User → API → AI Controller → Gemini Service → Google Gemini AI → AI Response

Use this flow to test the API end-to-end: first authenticate, then create workout data, search saved workouts, and finally request AI-powered suggestions and insights.

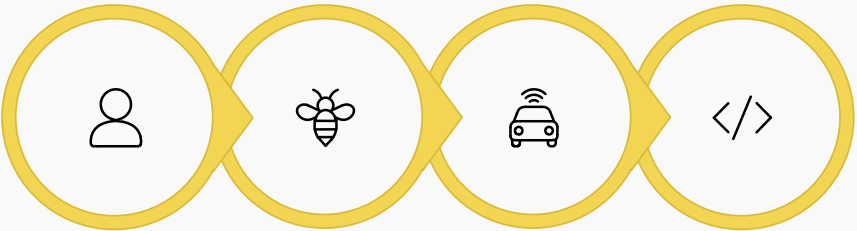
AI Workout Recommendation

POST /api/ai/workout-recommendation – JWT Required

```
{
  "age": 22,
  "fitnessGoal": "Weight Loss",
  "experience": "Beginner"
}
```

Response:

```
{
  "recommendation": "Start with 30 minutes
of brisk walking, 20 minutes of bodyweight
exercises, and stretching five days a week."
}
```



User Input

FitTrack API

Gemini AI

JSON
Recommendation

AI Fitness Insights

POST /api/ai/fitness-insights – JWT Required

```
{
  "totalWorkouts": 18,
  "averageDuration": 45,
  "totalCaloriesBurned": 6200
}
```

Response:

```
{
  "insight": "You have maintained a
consistent workout routine. Increasing
strength training may help achieve
your fitness goals faster."
}
```

Key Features



Secure Authentication

JWT-based authentication with bcrypt password hashing keeps user accounts protected and session access secure.



Comprehensive Workout Management

Create, view, update, and delete workout records with a complete CRUD workflow built for everyday fitness tracking.



Advanced Search

Use regex-based filtering to quickly find workouts by name, date, or other matching details without scrolling through records.



AI-Powered Recommendations

Google Gemini integration generates personalized workout recommendations based on user profile data and training goals.



Workout Analytics

AI-driven analysis reviews workout patterns and fitness statistics to surface practical insights and improvement opportunities.



Error Handling

Centralized middleware catches errors consistently, returning clean responses that make the API easier to debug and maintain.

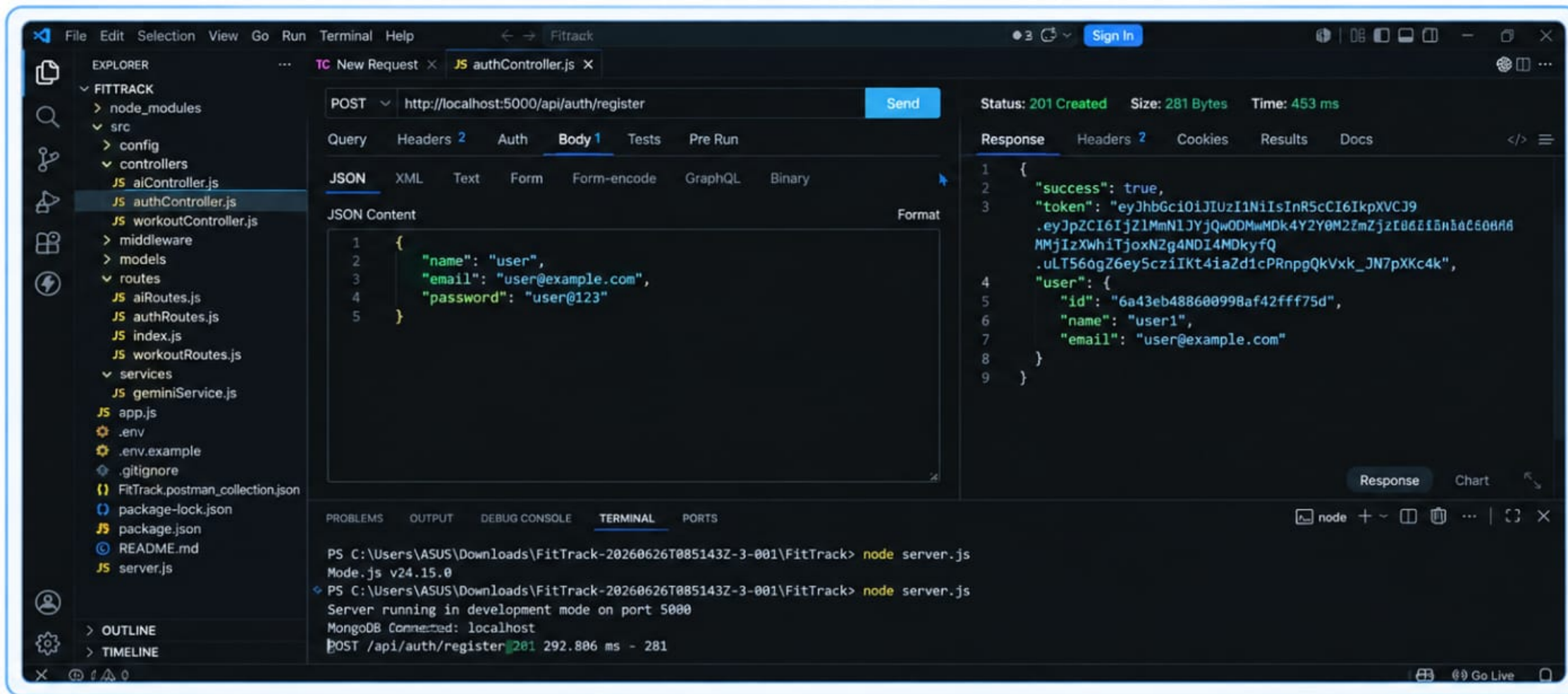


RESTful API Design

Standard REST conventions keep routes predictable, scalable, and easy to integrate with external clients and tools.



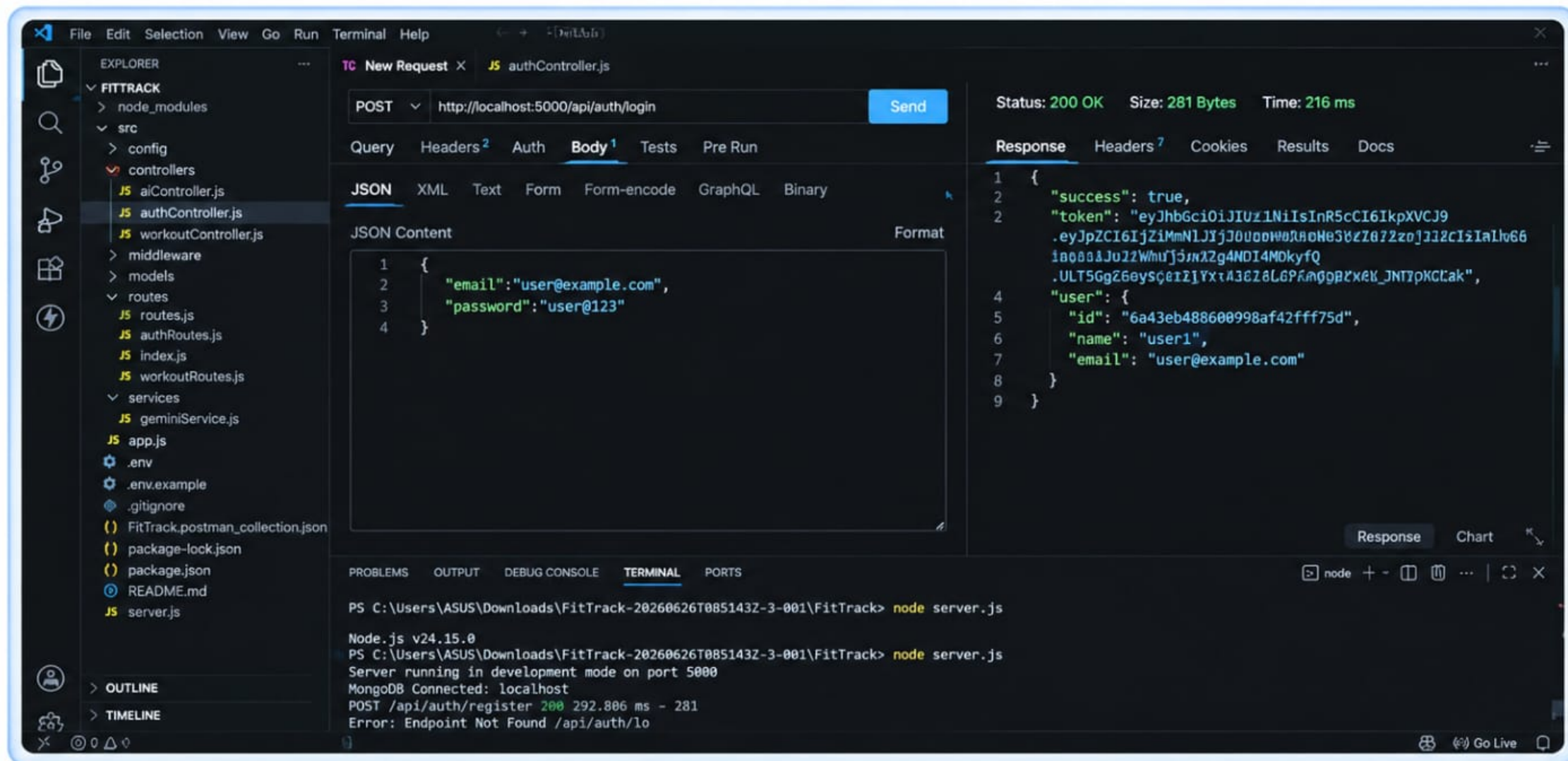
Stores a new user's details in the database using the /register endpoint.





Login User

Authenticate a user by sending login credentials and receiving a response.





Connects with the Google Gemini API to generate personalized workout recommendations.



User Input

Workout preferences,
goals, fitness level, etc.



Google Gemini API

Sends a prompt with user details to Gemini.



Generates Prompt

Creates a smart prompt based on user input.



AI Response

Returns personalized workout recommendation.



This feature connects to the Google Gemini API and uses it to generate a tailored prompt for workout recommendations.

MongoDB Connection

The screenshot shows a Visual Studio Code editor with a project named 'FitTrack'. The Explorer sidebar on the left shows the file structure, including 'controllers' and 'services' folders. The 'aiController.js' file is selected in the 'controllers' folder. The main editor displays the code for 'aiController.js', which includes a route for '/api/ai/workout-recommendation' that calls a 'geminiService' to generate a workout recommendation. The code is as follows:

```
1 const geminiService = require('../services/geminiService');
2
3 // @desc Get AI workout recommendation
4 // @route POST /api/ai/workout-recommendation
5 // @access Private
6 const getWorkoutRecommendation = async (req, res, next) => {
7   try {
8     const { age, fitnessGoal, experience } = req.body;
9
10    // Validate inputs
11    if (age === undefined || !fitnessGoal || !experience) {
12      return res.status(400).json({
13        success: false,
14        error: 'Please provide age, fitnessGoal, and experience',
15      });
16    }
17
18    // Call service to get recommendation from Gemini AI
19    const recommendation = await geminiService.generateWorkoutRecommendation(
20      age,
21      fitnessGoal,
22      experience
23    );
24
25    res.status(200).json({
26      recommendation,
27    });
28   } catch (error) {
```

The bottom of the screen shows the Terminal panel with the following output:

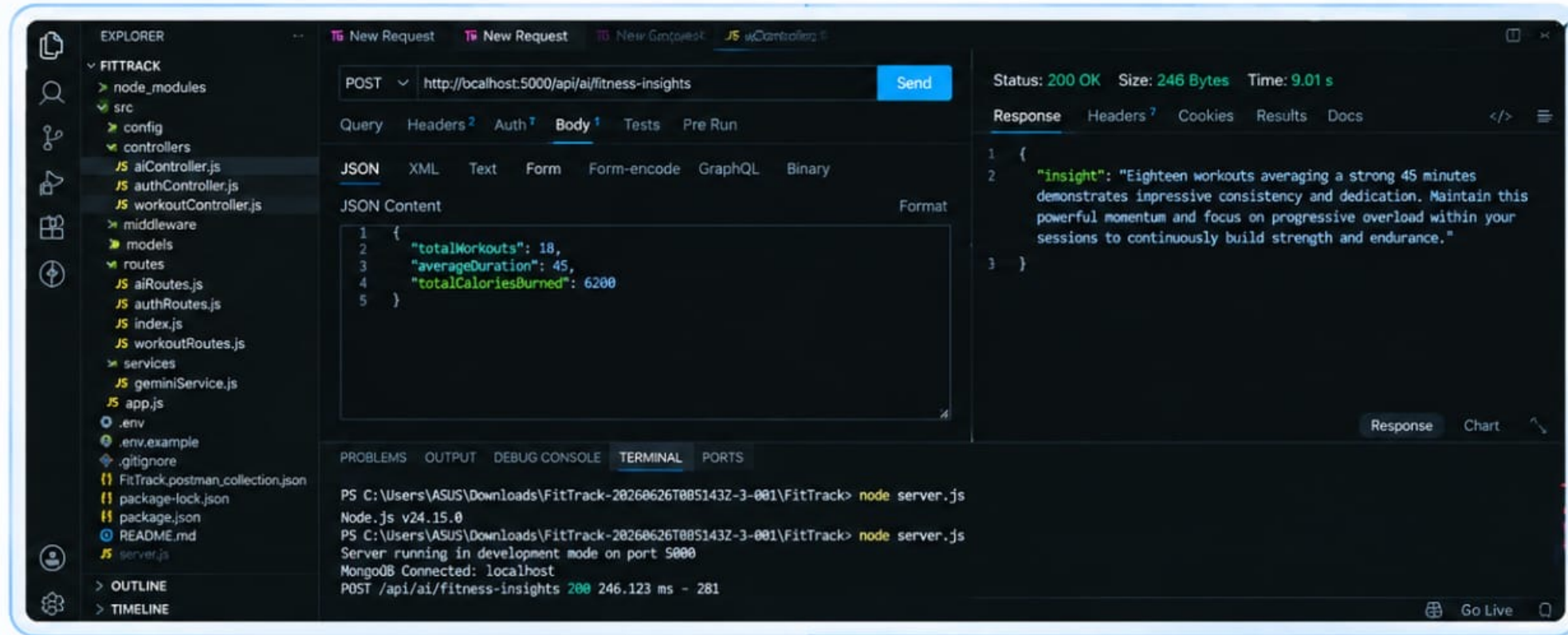
```
PS C:\Users\ASUS\Downloads\FitTrack-20260626T085143Z-3-001\FitTrack> node server.js
Mode.js v24.15.0
PS C:\Users\ASUS\Downloads\FitTrack-20260626T085143Z-3-001\FitTrack> node server.js
Server running in development mode on port 5000
MongoDB Connected: localhost
```

The status bar at the bottom indicates the file is 'aiController.js' at line 36, column 5, with 20 characters selected. The encoding is UTF-8 and the language is JavaScript.



Fitness-Insight

Shows quick fitness insights like total workouts, average duration and total calories burned from your workout data.



User Data

Workout details (duration, calories, frequency, etc.) are sent as an API request.



Google Gemini API

The data is sent to the Google Gemini API for analysis and insight generation.



AI Processing

Gemini analyzes the data and creates personalized fitness insights.



Insight Output

You get useful insights like total workouts, average duration and calories burned.



Smarter data. Better decisions. A healthier you.

Thank You



Daniya Fathima.B
Team Leader



Kathija Rayhana.R
Team Member



Aysha Mafazha.A.S
Team Member



Faiza Fathima.I
Team Member